T Bhanu Teja

192472259

C++ PROGRAMMING

UNIT V – VIRTUAL FUNCTIONS, POLYMORPHISM & EXCEPTION HANDLING

QUESTIONS WITH ANSWERS

Assessment Coverage

Section A contains five debugging questions with defect analysis, corrected C++ code, dry runs and outputs. Section B contains five structured pseudocode solutions with design notes, trace tables and expected results.

All explanatory text, code snippets and tables use Times New Roman. Explanatory paragraphs are justified and code is left-aligned for readability.

SECTION A – DEBUGGING QUESTIONS

Question 1: Non-Virtual Destructor Memory Leak in Polymorphic Deletion

Topic: Virtual Functions, Virtual Destructors, Dynamic Dispatch.

Defect and location: The Shape destructor is not virtual. Deleting a ComplexPolygon through Shape* without a virtual base destructor results in undefined behavior; the derived destructor is not guaranteed to run, so its vertex array may not be released.

Explanation: A virtual destructor enables dynamic dispatch during deletion. The derived destructor executes first, releasing derived resources, and then the base destructor executes. The vptr/vtable mechanism supports selecting the final overrider, including the destructor.

Corrected C++ Code:

```
#include <iostream>
using namespace std;

class Shape {
public:
    Shape() { cout << "Shape created." << endl; }
    virtual ~Shape() {
        cout << "Shape destroyed." << endl;
    }
    virtual void render() const {
        cout << "Rendering generic shape." << endl;
    }
};

class ComplexPolygon : public Shape {
private:
    double* vertexArray;
    int vertexCount;
public:
    ComplexPolygon(int count) : vertexCount(count) {
        vertexArray = new double[vertexCount * 2];
        cout << "Allocated " << vertexCount
             << " vertices on heap." << endl;
    }
    ~ComplexPolygon() override {
        cout << "Deallocating polygon vertex heap memory."
             << endl;
        delete[] vertexArray;
    }
    void render() const override {
        cout << "Rendering polygon with "
             << vertexCount << " vertices." << endl;
    }
```

```
};

int main() {
    Shape* shapePtr = new ComplexPolygon(1000);
    shapePtr->render();
    delete shapePtr;
    return 0;
}
```

Dry Run: The derived object is constructed after its Shape base. render() dispatches to ComplexPolygon. During delete, the derived destructor releases vertexArray before the Shape destructor runs.

Step	Operation	Result
1	Construct Shape base	Shape created.
2	Construct ComplexPolygon	Allocated 1000 vertices on heap.
3	Call render()	Rendering polygon with 1000 vertices.
4	delete shapePtr	Derived destructor releases array
5	Finish deletion	Shape destructor executes

Expected Output:

```
Shape created.
Allocated 1000 vertices on heap.
Rendering polygon with 1000 vertices.
Deallocating polygon vertex heap memory.
Shape destroyed.
```

C++ rule: Deleting a derived object through a base pointer when the base destructor is non-virtual is undefined behavior (unless a narrow special exception applies). A virtual destructor is the correct design for polymorphic base classes.

Question 2: Inverted Exception Catch Handlers & Unreachable Code

Topic: Exception Handling Mechanism, Catch Handler Ordering.

Defect and location: catch(const BankingException&) appears before catch(const InsufficientFundsException&). Since InsufficientFundsException derives from BankingException, the base handler catches it first, making the derived handler unreachable.

Explanation: Catch handlers are tested in source order. A derived exception can bind to a reference to its base class. Therefore, specific derived handlers must appear before general base handlers.

Corrected C++ Code:

```
#include <iostream>
#include <string>
using namespace std;

class BankingException {
protected:
    string msg;
public:
    BankingException(string m) : msg(m) {}
    virtual string getMessage() const { return msg; }
    virtual ~BankingException() = default;
};

class InsufficientFundsException : public BankingException {
private:
    double shortfall;
public:
    InsufficientFundsException(string m, double sf)
        : BankingException(m), shortfall(sf) {}
    double getShortfall() const { return shortfall; }
};

void processWithdrawal(double balance, double amount) {
    if (amount > balance)
        throw InsufficientFundsException(
            "Account overdrawn!", amount - balance);
}

int main() {
    try {
        processWithdrawal(500.0, 750.0);
    }
    catch (const InsufficientFundsException& e) {
        cout << "Shortfall of: $" << e.getShortfall() << endl;
    }
    catch (const BankingException& e) {
        cout << "General Banking Error: "
             << e.getMessage() << endl;
    }
    catch (...) {
        cout << "Unknown banking error." << endl;
    }
    return 0;
}
```

Dry Run: The attempted withdrawal is greater than the balance. processWithdrawal throws InsufficientFundsException. The first handler matches that exact derived type, so the base and catch-all handlers are skipped.

Step	Execution	Result
1	processWithdrawal(500,750)	750 > 500
2	throw InsufficientFundsException	Shortfall = 250
3	Derived catch handler	Handles exception
4	Remaining handlers	Skipped

Expected Output:

```
Shortfall of: $250
```

Compiler note: Compilers commonly warn that the derived handler is shadowed by the earlier base handler; the exact diagnostic wording depends on the compiler.

Question 3: Exception Rethrowing Slicing Bug (throw e; vs. throw;)

Topic: Throwing, Catching and Rethrowing Exceptions, Object Slicing.

Defect and location: Inside intermediateAuditor(), throw e; creates and throws a copy using the static type DatabaseError. The derived TimeoutError portion, including timeoutMs, is sliced away.

Explanation: A bare throw; rethrows the currently handled exception object and preserves its dynamic type. throw e; throws a new object based on the static type of the expression e.

Corrected C++ Code:

```
#include <iostream>
#include <exception>
#include <string>
using namespace std;

class DatabaseError : public exception {
protected:
    string errorDetails;
public:
    DatabaseError(string msg) : errorDetails(msg) {}
    const char* what() const noexcept override {
        return errorDetails.c_str();
    }
};

class TimeoutError : public DatabaseError {
public:
```

```
    int timeoutMs;
    TimeoutError(string msg, int ms)
        : DatabaseError(msg), timeoutMs(ms) {}
};

void executeQuery() {
    throw TimeoutError(
        "Query timed out connecting to shard 3", 5000);
}

void intermediateAuditor() {
    try {
        executeQuery();
    }
    catch (const DatabaseError& e) {
        cout << "[AUDIT LOG]: Intercepted DB error."
             << endl;
        throw; // Preserve original exception type
    }
}

int main() {
    try {
        intermediateAuditor();
    }
    catch (const TimeoutError& e) {
        cout << "Client caught TimeoutError! Duration: "
             << e.timeoutMs << " ms" << endl;
    }
    catch (const DatabaseError& e) {
        cout << "Client caught generic DatabaseError: "
             << e.what() << endl;
    }
    return 0;
}
```

Dry Run: executeQuery throws TimeoutError. intermediateAuditor catches it through a DatabaseError reference, logs the event and uses bare throw;. The original TimeoutError reaches main(), where the specific handler catches it.

Step	Execution	Control / Result
1	executeQuery()	Throws TimeoutError(5000)
2	DatabaseError reference catch	Catches derived exception through base reference
3	Print audit log	Audit message displayed
4	throw;	Rethrows original

		TimeoutError
5	main() TimeoutError handler	Reads timeoutMs = 5000

Expected Output:

```
[AUDIT LOG]: Intercepted DB error.
Client caught TimeoutError! Duration: 5000 ms
```

Question 4: Attempting to Instantiate an Incomplete Abstract Class

Topic: Pure Virtual Functions, Abstract Classes, Runtime Interfaces.

Defect and location: VideoPacket implements serialize() but does not implement the pure virtual getPacketSize(). Therefore, VideoPacket remains abstract and cannot be instantiated at packet(1042).

Explanation: A class is abstract if it has at least one pure virtual function that has no final overrider. An abstract class cannot be instantiated directly. A derived class becomes concrete only after implementing every inherited pure virtual function.

Corrected C++ Code:

```
#include <iostream>
using namespace std;

class NetworkPacket {
public:
    virtual ~NetworkPacket() = default;
    virtual void serialize() = 0;
    virtual int getPacketSize() = 0;
};

class VideoPacket : public NetworkPacket {
private:
    int frameID;
public:
    VideoPacket(int id) : frameID(id) {}

    void serialize() override {
        cout << "Serializing H.264 Video Frame: "
             << frameID << endl;
    }

    int getPacketSize() override {
        return 1024; // Example packet size in bytes
    }
};
```

```
int main() {
    VideoPacket packet(1042);
    packet.serialize();
    cout << "Packet size: "
         << packet.getPacketSize() << " bytes" << endl;
    return 0;
}
```

Dry Run: VideoPacket now overrides both pure virtual functions. It is a concrete class, so the object can be created. serialize() prints the frame identifier and getPacketSize() returns the example size.

Step	Operation	Result
1	Create VideoPacket(1042)	Object created
2	packet.serialize()	Serializing H.264 Video Frame: 1042
3	packet.getPacketSize()	1024

Expected Output:

```
Serializing H.264 Video Frame: 1042
Packet size: 1024 bytes
```

Note: The source does not specify a real encoded packet size. The corrected example uses 1024 bytes as an illustrative value.

Question 5: Pass-by-Value Object Slicing Suppressing Dynamic Binding

Topic: Runtime Polymorphism, Early vs. Late Binding, Object Slicing.

Defect and location: triggerEncounter(Entity e) accepts the polymorphic object by value. Passing Dragon copies only its Entity base subobject into a separate Entity object, so the Dragon part is sliced away and Entity::attack() executes.

Explanation: Runtime dispatch depends on the dynamic type of the object. A base reference or pointer refers to the original Dragon object and preserves its dynamic type. A by-value base parameter creates a separate base object.

Corrected C++ Code:

```
#include <iostream>
using namespace std;

class Entity {
public:
    virtual ~Entity() = default;
    virtual void attack() const {
        cout << "Entity performs generic scratch."
```

```
            << endl;
    }
};

class Dragon : public Entity {
public:
    void attack() const override {
        cout << "Dragon breathes scorching fire!"
            << endl;
    }
};

void triggerEncounter(const Entity& e) {
    e.attack();
}

int main() {
    Dragon boss;
    triggerEncounter(boss);
    return 0;
}
```

Dry Run: boss is a Dragon object. triggerEncounter receives a reference to the same object. The virtual call uses the object's dynamic type, so Dragon::attack() is selected.

Step	Execution	Result
1	Create Dragon boss	Dynamic type is Dragon
2	Pass boss as const Entity&	No copy or slicing
3	Call e.attack()	Virtual dispatch selects Dragon::attack()

Expected Output:

```
Dragon breathes scorching fire!
```

vptr note: A by-value Entity copy is an Entity object, so its dynamic type is Entity and its virtual calls resolve accordingly. With a reference, the original Dragon object's dynamic type remains available for virtual dispatch.

SECTION B – PSEUDO-CODE WRITING QUESTIONS

The solutions below follow the source assessment's class names, functional requirements and pseudocode conventions.

Question 1: Universal Payment Gateway with Runtime Polymorphism

Topic: Pure Virtual Functions, Runtime Polymorphism, Abstract Base Classes.

Design Overview: PaymentGateway defines a common abstract interface. CreditCardGateway and CryptoGateway implement authorization and capture. Main stores the selected concrete object through a PaymentGateway pointer, enabling late binding.

Structured Pseudocode:

```
ABSTRACT CLASS PaymentGateway
   PURE VIRTUAL METHOD Authorize(amount AS REAL)
      RETURNS BOOLEAN
   PURE VIRTUAL METHOD Capture() RETURNS BOOLEAN
   VIRTUAL DESTRUCTOR()
END CLASS

CLASS CreditCardGateway EXTENDS PaymentGateway
   DECLARE cardNumber, cvv AS STRING
   OVERRIDE METHOD Authorize(amount) RETURNS BOOLEAN
      IF LENGTH(cardNumber) = 16 AND LENGTH(cvv) = 3
         PRINT "Card authorization successful"
         RETURN TRUE
      ELSE
         PRINT "Invalid card details"
         RETURN FALSE
      END IF
   END METHOD
   OVERRIDE METHOD Capture() RETURNS BOOLEAN
      PRINT "Credit card payment captured"
      RETURN TRUE
   END METHOD
END CLASS

CLASS CryptoGateway EXTENDS PaymentGateway
   DECLARE walletHash AS STRING
   OVERRIDE METHOD Authorize(amount) RETURNS BOOLEAN
      IF walletHash STARTS WITH "0x" THEN
         PRINT "Wallet verified"
         IF GAS_FEE_VALID(amount) THEN
            PRINT "Gas fee validated"
            RETURN TRUE
         END IF
      END IF
      PRINT "Crypto authorization failed"
```

```
        RETURN FALSE
    END METHOD
    OVERRIDE METHOD Capture() RETURNS BOOLEAN
        PRINT "Cryptocurrency payment captured"
        RETURN TRUE
    END METHOD
END CLASS

MAIN
    DECLARE gatewayPtr AS POINTER TO PaymentGateway
    gatewayPtr <- NEW CryptoGateway("0xABCD1234")
    IF gatewayPtr->Authorize(1500.00) THEN
        gatewayPtr->Capture()
    END IF
    DELETE gatewayPtr
END MAIN
```

Dry Run – $1,500 Cryptocurrency Payment:

Step	Action	Result
1	Create CryptoGateway	Base pointer refers to CryptoGateway
2	Authorize(1500.00)	CryptoGateway override executes
3	Check wallet prefix	Wallet begins with 0x
4	Validate gas fee	Assumed valid in this example
5	Capture()	Crypto payment captured
6	Delete base pointer	Virtual destructor cleans up derived object

Expected Result:

```
Wallet verified
Gas fee validated
Cryptocurrency payment captured
```

Assumption: GAS_FEE_VALID(1500.00) returns TRUE for this trace; the assessment does not define a numerical gas-fee policy.

Question 2: Resilient Banking Engine with Nested Try-Catch & Rethrowing

Topic: Custom Exception Hierarchy, Nested try-catch, Rethrowing.

Design Overview: TransactionException is the base exception. AccountFrozenException and DailyLimitExceededException represent specific failures. DebitLedger throws the appropriate

exception. AuditTransactionWrapper catches the base type, logs it and rethrows the active exception using RETHROW.

Structured Pseudocode:

```
CLASS TransactionException
    DECLARE reason AS STRING
    CONSTRUCTOR(message)
        reason <- message
    END CONSTRUCTOR
END CLASS

CLASS AccountFrozenException EXTENDS TransactionException
    CONSTRUCTOR(message)
        BASE TransactionException(message)
    END CONSTRUCTOR
END CLASS

CLASS DailyLimitExceededException EXTENDS TransactionException
    DECLARE excessAmount AS REAL
    CONSTRUCTOR(message, excess)
        BASE TransactionException(message)
        excessAmount <- excess
    END CONSTRUCTOR
END CLASS

FUNCTION DebitLedger(amount, accountFrozen, dailyTotal)
    IF accountFrozen = TRUE THEN
        THROW NEW AccountFrozenException("Account is frozen")
    END IF
    IF dailyTotal + amount > 50000 THEN
        excess <- dailyTotal + amount - 50000
        THROW NEW DailyLimitExceededException(
            "Daily withdrawal limit exceeded", excess)
    END IF
    PRINT "Debit successful"
END FUNCTION

FUNCTION AuditTransactionWrapper(amount, frozen, dailyTotal)
    TRY
        DebitLedger(amount, frozen, dailyTotal)
    CATCH (e AS TransactionException)
        PRINT "SECURITY AUDIT: ", e.reason
        RETHROW
    END TRY
END FUNCTION
```

```
MAIN
   TRY
      AuditTransactionWrapper(1000, TRUE, 10000)
   CATCH (e AS AccountFrozenException)
      PRINT "Transaction denied: account is frozen"
   CATCH (e AS DailyLimitExceededException)
      PRINT "Transaction denied: daily limit exceeded"
   CATCH (e AS TransactionException)
      PRINT "Transaction failed: ", e.reason
   END TRY
END MAIN
```

Dry Run – $1,000 Withdrawal on Frozen Account:

Step	Control Flow	Result
1	Main calls AuditTransactionWrapper	Withdrawal request = 1000
2	Wrapper calls DebitLedger	accountFrozen = TRUE
3	DebitLedger throws AccountFrozenException	Control leaves DebitLedger
4	Wrapper catches TransactionException	Security audit log is written
5	RETHROW	Same active exception propagates
6	Main catches AccountFrozenException	Client is notified; later catches skipped

Expected Result:

```
SECURITY AUDIT: Account is frozen
Transaction denied: account is frozen
```

Question 3: Polymorphic Game Engine with Virtual Destructors

Topic: Virtual Functions, Virtual Destructors, Polymorphic Object Lifecycles.

Design Overview: GameObject is an abstract base class with pure virtual Update() and Render() methods and a virtual destructor. ParticleSystem implements the methods and owns a dynamic particle-coordinate array. Deleting through GameObject* invokes the derived destructor before the base destructor.

Structured Pseudocode:

```
ABSTRACT CLASS GameObject
   PURE VIRTUAL METHOD Update(deltaTime AS REAL)
   PURE VIRTUAL METHOD Render()
   VIRTUAL DESTRUCTOR()
```

```
        PRINT "GameObject destroyed"
    END DESTRUCTOR
END CLASS

CLASS ParticleSystem EXTENDS GameObject
    DECLARE particleCoordinates AS POINTER TO REAL
    DECLARE particleCount AS INTEGER

    CONSTRUCTOR(count)
        particleCount <- count
        particleCoordinates <- NEW ARRAY OF REAL[particleCount * 2]
        PRINT "Particle memory allocated"
    END CONSTRUCTOR

    OVERRIDE METHOD Update(deltaTime)
        PRINT "Particles updated by ", deltaTime
    END METHOD

    OVERRIDE METHOD Render()
        PRINT "Particle system rendered"
    END METHOD

    DESTRUCTOR()
        DELETE ARRAY particleCoordinates
        PRINT "Particle memory released"
    END DESTRUCTOR
END CLASS

MAIN
    DECLARE sceneEntities AS ARRAY OF POINTER TO GameObject[1]
    sceneEntities[0] <- NEW ParticleSystem(100)

    FOR EACH entity IN sceneEntities
        entity->Update(0.016)
        entity->Render()
    END FOR

    FOR i <- 0 TO 0
        DEALLOCATE sceneEntities[i]
    END FOR
END MAIN
```

Dry Run / Cleanup Trace:

Step	Operation	Result
1	Allocate ParticleSystem	Particle memory allocated

2	Update(0.016)	Particles updated by 0.016
3	Render()	Particle system rendered
4	DEALLOCATE base pointer	Virtual dispatch starts derived destructor
5	ParticleSystem destructor	Particle memory released
6	GameObject destructor	GameObject destroyed

Expected Result:

```
Particle memory allocated
Particles updated by 0.016
Particle system rendered
Particle memory released
GameObject destroyed
```

Destructor order: derived destructor first, then base destructor. This is why the base destructor must be virtual when objects are deleted polymorphically.

Question 4: File Stream Pipeline with Stack Unwinding & Catch-All

Topic: Stack Unwinding, Resource Acquisition (RAII), Catch-All Handler.

Design Overview: ResourceGuard acquires a resource in its constructor and releases it in its destructor. IngestDataStream creates it as an automatic local object. If checksum validation throws, stack unwinding destroys the guard before control reaches Main's catch handler.

Structured Pseudocode:

```
CLASS ResourceGuard
    DECLARE handle
    CONSTRUCTOR(fileName)
        handle <- OPEN_RESOURCE(fileName)
        PRINT "Resource acquired"
    END CONSTRUCTOR
    DESTRUCTOR()
        CLOSE_RESOURCE(handle)
        PRINT "Resource released"
    END DESTRUCTOR
END CLASS

CLASS CRCValidationException
    DECLARE expectedChecksum, calculatedChecksum
    CONSTRUCTOR(expected, calculated)
        expectedChecksum <- expected
        calculatedChecksum <- calculated
    END CONSTRUCTOR
END CLASS
```

```
FUNCTION IngestDataStream(fileName)
    DECLARE guard AS ResourceGuard(fileName)
    PRINT "Reading data blocks"
    expected <- 500
    calculated <- 450
    IF expected <> calculated THEN
        THROW NEW CRCValidationException(expected, calculated)
    END IF
    PRINT "Data accepted"
END FUNCTION

MAIN
    TRY
        IngestDataStream("mission-data.bin")
    CATCH (e AS CRCValidationException)
        PRINT "CRC validation failed"
        PRINT "Expected: ", e.expectedChecksum
        PRINT "Calculated: ", e.calculatedChecksum
    CATCH (...)
        PRINT "Unknown ingestion failure"
    END TRY
END MAIN
```

Dry Run / Stack Unwinding:

Order	Event	Explanation
1	ResourceGuard constructed	Resource handle acquired
2	Read data blocks	Checksum values calculated
3	CRC mismatch detected	CRCValidationException thrown
4	Stack unwinding begins	Local automatic objects are destroyed
5	ResourceGuard destructor runs	Handle is closed/released
6	Main catch executes	CRC error details are displayed

Expected Result:

```
Resource acquired
Reading data blocks
Resource released
CRC validation failed
Expected: 500
Calculated: 450
```

The catch-all handler is the final fallback and is not executed for the CRCValidationException because the specific handler matches it.

Question 5: Scientific Telemetry Calibration with noexcept Contracts

Topic: Specifying Exceptions, noexcept Contracts, Boundary Assertions.

Design Overview: CalibrateThermalSensors may throw OutOfBoundsException when the calculated temperature is outside the permitted interval. CommitTelemetry is noexcept and catches all internal failures so that no exception escapes it. Main catches calibration errors.

Structured Pseudocode:

```
CLASS OutOfBoundsException
    DECLARE measuredTemperature AS REAL
    CONSTRUCTOR(temp)
        measuredTemperature <- temp
    END CONSTRUCTOR
END CLASS

FUNCTION CalibrateThermalSensors(rawVoltage)
    NOEXCEPT(FALSE)
    temperature <- CONVERT_VOLTAGE_TO_TEMPERATURE(rawVoltage)
    IF temperature < -273.15 OR temperature > 1500.0 THEN
        THROW NEW OutOfBoundsException(temperature)
    END IF
    RETURN temperature
END FUNCTION

FUNCTION CommitTelemetry(finalTemp) NOEXCEPT
    TRY
        WRITE_TO_AVIONICS_BUS(finalTemp)
        PRINT "Telemetry committed"
    CATCH (...)
        PRINT "Commit failure suppressed"
    END TRY
END FUNCTION

MAIN
    TRY
        temperature <- CalibrateThermalSensors(FAULTY_VOLTAGE)
        CommitTelemetry(temperature)
    CATCH (e AS OutOfBoundsException)
        PRINT "Calibration rejected: temperature out of bounds"
        PRINT "Measured temperature: ", e.measuredTemperature
    CATCH (...)
        PRINT "Unexpected telemetry error"
```

```
    END TRY
END MAIN
```

Dry Run – Faulty Reading Produces -300.0°C:

Step	Operation	Result
1	Call calibration function	Calculated temperature = -300.0°C
2	Check lower boundary	-300.0 < -273.15
3	Throw OutOfBoundsException	Control returns to Main's handler
4	CommitTelemetry()	Not called because calibration failed
5	Main catches exception	Displays rejected temperature

Expected Result:

```
Calibration rejected: temperature out of bounds
Measured temperature: -300
```

noexcept note: If an exception escapes a noexcept function, C++ calls std::terminate. The pseudocode catches all exceptions inside CommitTelemetry to prevent propagation. The assessment does not specify the voltage-to-temperature conversion formula; the trace assumes the stated calculated value of -300.0°C.